1. Eclipse->File->New->Others->Maven->Maven Project
2. Uncheck the default workspace location
3. Select maven archetype quickstart
4. Groupid: ToolsQA, artifcatId : myproject
5. src/main/java ->new->package->com.sdet
6. com.sdet->new->class->app
7. src/main/resources->new->file->config.properties(username & password)
8. src/test/java->new->package->com.sdet
9. com.sdet->new->class->apptest
10. Click on pom.xml->run as->maventest

```
import java.util.ResourceBundle;
public class MyApp
{
public int userlogin(string inuser,string inpwd)
{
ResourceBundle rb= ResourceBundle.getBundle(“config”);
String username=rb.getstring(“username”);
String password=rb.getstring(“password”);
If(inuser.equals(username)&& inpassword(password))
        return 1;
else
        return 0;
}
username=abc
password=abc@1234
package myproject;
import org.testng.Assert;
import org.testng.annotations.Test;
import com.myproject.app;
public class apptest {
   @Test
   public void testlogin1()
   {
            app myapp=new app();
            Assert.assertEquals(0,myapp.userlogin("abc","abc1234"));
   }
@Test
public void testlogin2()
{
app myapp=new app();
Assert.assertEquals(1,myapp.userlogin("abc","abc@1234"));
}
}
<dependencies>
  <!-- https://mvnrepository.com/artifact/org.testng/testng -->
<dependency>
   <groupId>org.testng</groupId>
   <artifactId>testng</artifactId>
   <version>7.5.1</version>
   <scope>test</scope>
</dependency>
  </dependencies>
```

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def hello():
    return "Hello from App 1!! Kubernetes, also known as K8s,is an open source system for automating deployment, scaling, and management of containerized applications"
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirement.tx flask==3.0.0

Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

- **docker build -t myapp .**
- **docker push -p myapp**

Kubernetes Deployment (deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hw-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
      - name: hw-container
        image: chethanaravi/app1-k8s:latest
        ports:
        - containerPort: 5000
```

Kubernetes Service (service.yam0l)

```
apiVersion: v1
kind: Service
metadata:
  name: hello-world
spec:
  type: NodePort
  selector:
    app: hello-world
  ports:
    - port: 5000
```

```
  targetPort: 5000
```

- **kubectl apply -f deployment.yaml**
- **kubectl apply -f service.yaml**
- **kubectl get pods**
- **kubectl get svc**

Step 1: Create a Project Folder with name as terraform on your desktop.

Step 2: Create the main.tf File

Create a file named main.tf and paste the following code:

```
provider "aws" {
  region = "us-west-2"
}
resource "aws_instance" "ec2_machine" {
 ami = "ami-07b0c09aab6e66ee9"
 instance_type = "t2.micro"
                                                //  count=4
 tags ={
   Name = "Terra EC2"
 }
}
```

Step 3: Initialize Terraform: execute following command on VS code terminal :

```
terraform init
```

This sets up Terraform in the folder and downloads necessary provider plugins.

Step 4: Preview the Plan

```
terraform plan
```

This shows what Terraform will create.

Step 5: Apply the Configuration

```
terraform apply
```

Step 6: Verify

Go to your **AWS Console → EC2 → Instances**, and you'll see the instance running.

Step 7: Destroy When Done (to avoid charges)

```
terraform destroy
```